

Intermediate (build the loop yourself) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

```
class MaxIterationsExceeded(Exception):
    pass

def run_agent(task, max_iterations=5):
    messages = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
        else:
            return message.content

    raise MaxIterationsExceeded("Agent ran out of steps without an answer: " + task)
```

This version works correctly for one tool that always succeeds. It's missing type hints, and `run_tool()` here is assumed to never raise — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — one tool, minimal structure

```
def run_tool(call) -> str:
    import json
    try:
        args = json.loads(call.function.arguments)
        result = get_weather(**args)
        return str(result)
    except Exception as e:
        return f"Error: {e}"

def run_agent(task: str, max_iterations: int = 5) -> str:
    messages: list[dict] = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[weather_tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
        else:
            return message.content

    raise MaxIterationsExceeded(f"No answer after {max_iterations} steps")
```

Approach 2 — logging every step, and returning the full trace

```
from dataclasses import dataclass, field
```

```

class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        self.steps = steps
        super().__init__(message)

@dataclass
class AgentResult:
    final_answer: str
    steps: list[dict] = field(default_factory=list)

def run_agent(task: str, max_iterations: int = 5) -> AgentResult:
    messages: list[dict] = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[weather_tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
                steps.append({
                    "step": step,
                    "tool": call.function.name,
                    "arguments": call.function.arguments,
                    "result": result,
                })
        else:
            return AgentResult(final_answer=message.content, steps=steps)

    raise MaxIterationsExceeded(f"No answer after {max_iterations} steps", steps)

```

Difference from Basic: both add full type hints (`task: str` , `-> str` / `-> AgentResult`).

Approach 2 additionally records each step's tool, arguments, and result as it happens, and returns them alongside the answer — once the final answer is wrong, you have a real trail to look at instead of re-running with print statements sprinkled in by hand.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — every tool call handled, every tool failure recovered from

```
import json
from dataclasses import dataclass, field

class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        self.steps = steps
        super().__init__(message)

@dataclass
class AgentResult:
    final_answer: str
    steps: list[dict] = field(default_factory=list)

TOOLS_BY_NAME = {
    "get_weather": get_weather,
    "celsius_to_fahrenheit": celsius_to_fahrenheit,
}

def run_tool(call) -> str:
    function = TOOLS_BY_NAME.get(call.function.name)
    if function is None:
        return f"Error: unknown tool {call.function.name}"
    try:
        args = json.loads(call.function.arguments)
        result = function(**args)
        return str(result)
    except Exception as e:
        return f"Error: {e}"

def run_agent(task: str, max_iterations: int = 5) -> AgentResult:
    messages: list[dict] = [{"role": "user", "content": task}]
    steps: list[dict] = []
```

```

for step in range(max_iterations):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        tools=[weather_tool_schema, fahrenheit_tool_schema],
    )
    message = response.choices[0].message

    if message.tool_calls:
        messages.append(message)
        for call in message.tool_calls:
            result = run_tool(call)
            messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
            steps.append({
                "step": step,
                "tool": call.function.name,
                "arguments": call.function.arguments,
                "result": result,
            })
    else:
        return AgentResult(final_answer=message.content, steps=steps)

raise MaxIterationsExceeded(f"No answer after {max_iterations} steps", steps)

```

Expected behavior if `get_weather` raises on a bad city name: `run_tool()` catches it, and the loop's next round sees `{"role": "tool", "content": "Error: city not found"}` as the Observation — the model gets a real chance to react (retry, ask for clarification, or say it can't answer), instead of the whole program crashing and every already-completed step being lost.

Approach 2 — a step-limit test that actually proves the guarantee

```

def make_adversarial_tool_call_response():
    """A stand-in model response that always asks for another tool call,
    used to prove the loop truly stops at max_iterations."""
    ...

def test_max_iterations_is_enforced(monkeypatch):
    monkeypatch.setattr(client.chat.completions, "create", make_adversarial_tool_call_response)
    try:
        run_agent("a task designed to never satisfy the model", max_iterations=3)
        assert False, "expected MaxIterationsExceeded"
    except MaxIterationsExceeded as e:
        assert len(e.steps) == 3

```

Why this test matters: Core Concepts' claim — "a hard limit set in your code guarantees the loop stops" — is only true if something actually tests it. A test that forces the model to always request another tool call, and asserts `MaxIterationsExceeded` is raised at exactly `max_iterations` steps, is what turns "should stop" into "provably does stop." This is a preview of the Failure exercise below, written as an automated test instead of something you watch happen once.

Difference from Intermediate: Intermediate Approach 2 logs steps and handles one tool with a bare `try/except`. Advanced Approach 1 generalizes to a real multi-tool dispatch table (`T00LS_BY_NAME`) so `run_tool()` works for any registered tool, not just `get_weather` by name — this is what the Build Task's "3+ tools" requirement actually needs. Approach 2 adds the thing neither Intermediate approach has at all: proof, via an automated test, that the step limit genuinely holds under adversarial conditions, not just "it looked fine when I ran it once."

Which one should you actually write? Advanced Approach 1's shape — a dispatch table, a `run_tool()` that never lets an exception escape, and a returned `steps` log — is what the Build Task's `agent.py` should look like. Approach 2's kind of test belongs in `test_agent.py`, matching the Test Cases table's "task designed to loop forever" row exactly. Intermediate Approach 1 is genuinely fine for a single quick script with one trusted tool; reach for the Advanced shape the moment you have more than one tool, or any tool that touches the network, a file, or anything else that can fail.